



30-Day Android Challenge

Mastering Android Programming Interviews Globally
(Junior-Mid-Senior-Architect-Level Deconstruction 📱)

Day 10:

Testing Strategies

(Unit, Integration, UI Testing)

Part 2 of 2



Esmaeeil Moradi

@esmaeeil-moradi700

Day 10



of #30DayAndroidChallenge: Testing Strategies (Unit, Integration, UI Testing)

Greetings and a warm welcome to all of you, fellow developers and architects of the Android world! 🚀

We are incredibly thrilled to have you join us on this exciting and challenging journey of mobile development. You are the beating heart of innovation and progress in this field, and every line of code you write shapes the world for millions of users.

This is a home for knowledge, experience, and growth; a place where we can collectively push the boundaries of creativity.

Welcome to our community, future builders of Android applications! ✨

As applications grow in complexity and team size, a robust and scalable architecture becomes paramount. This session is designed to move beyond fundamental patterns, exploring advanced architectural considerations, effective multi-module strategies, and best practices for maintaining a healthy codebase in vast projects. My goal is to equip you with the insights necessary to not only ace top-tier interviews but also to architect and lead the development of enterprise-grade, highly maintainable Android applications in the real world. "Building Scalable Foundations for Tomorrow's Apps" is our motto for today.



Esmaeel Moradi
@esmaeel-moradi700



Architect Level



Esmaeel Moradi
@esmaeeil-moradi700

Architect a CI/CD (Continuous Integration/Continuous Delivery) pipeline that manages the automated execution of various tests (unit, integration, UI, Lint Checks) and the building of different application variants.

Architecting a robust CI/CD pipeline for a complex Android application is the backbone of modern software development, ensuring quality, speed, and reliability of software delivery.



Esmaeel Moradi
@esmaeel-moradi700

CI/CD Pipeline Stages and Components:



1. Continuous Integration (CI):

- **Goal:** Continuously integrate developers' code changes into a shared repository and automatically validate each change.
- **Trigger:** Whenever code is pushed to the main branch (e.g., main or develop) or a Pull Request (PR) is created/updated.
- **Stages:**
 - Checkout Code: Get the latest code from the version control system (Git).
 - Install Dependencies: Download and install all Gradle dependencies.
 - Lint Checks / Static Analysis:
 - Execute static code analysis tools like Android Lint, detekt (for Kotlin code quality), ktlint (for code formatting). This stage should be fast and catch code style issues and simple errors before running tests.
 - Architect's Insight: Custom Lint rules can be used to enforce team-specific Design System standards or architectural patterns.



Esmaeel Moradi

@esmaeel-moradi700



- **Unit Tests:**

- Execute all unit tests (./gradlew testDebugUnitTest).
- Benefit: Extremely fast and provide immediate feedback. If this stage fails, there's no need to continue the pipeline to slower stages.
- Collect Code Coverage reports (e.g., with JaCoCo).

- **Integration Tests (Local):**

- Integration tests that don't require a device/emulator (e.g., Repository with in-memory database, or network layer with MockWebServer).
- These tests are still relatively fast.

- **Build Artifacts (Debug/Release):**

- Build Debug and Release versions of the APK/AAB (depending on App Variants).
- ./gradlew assembleDebug and ./gradlew assembleRelease.



Esmaeel Moradi

@esmaeel-moradi700

2. Continuous Delivery (CD):



- **Goal:** Ensure that production-ready code is automatically delivered to subsequent environments (e.g., Test Environment, Staging, Production).
- **Trigger:** Typically after all CI stages succeed on the main branch, or manually for releases.
- **Stages:**
 - Instrumented Tests (Integration & UI Tests):
 - Execute all instrumented tests (./gradlew connectedCheck).
 - Execution Environment: On Android emulators (using AVD Manager in CI) or device farms like Firebase Test Lab, AWS Device Farm.
 - Architect's Insight: To optimize time, UI tests can be parallelized and run on multiple emulators/devices simultaneously. Test Sharding (dividing tests among multiple devices) is essential for large projects.
 - Test Reporting: Collect JUnit XML reports and display them in the CI tool's UI (e.g., Jenkins, GitHub Actions, GitLab CI).



Esmaeel Moradi

@esmaeel-moradi700

- **App Variant Builds:**

- App Variants Concept: Using Product Flavors and Build Types in Gradle to build different versions of the app (e.g., freeDebug, proRelease, stagingDebug, productionRelease).
- Management in CI/CD: The pipeline must be capable of building specific App Variants based on the branch (e.g., main for productionRelease) or tag.
- Example: assembleProductionRelease or bundleStagingDebug.
- Architect's Insight: Correct management of applicationId, versionCode, versionName, and resources (e.g., API Keys) for each Variant is crucial. This should be managed via buildConfigField or resValue in Gradle.



Esmaeel Moradi

@esmaeel-moradi700



- **Signing:**
 - Signing the APK/AAB with the corresponding Keystore (for Release Builds).
 - Security: The Keystore must be securely stored in the CI/CD and injected into the pipeline only when needed (e.g., via Secret Management).
- **Deploy to Internal Channels / Staging:**
 - Automated deployment of Debug/Staging versions to internal testing channels (e.g., Firebase App Distribution, Microsoft App Center, TestFlight).
 - Notification to QA/Testers team to start manual tests.
- **Deploy to Production:**
 - After QA approval, automated (or with manual approval) release to Google Play Store (using Play Console API) or other app stores.
 - Architect's Insight: Implement gradual Rollout (e.g., 5% of users initially) to reduce the risk of releasing bugs to all users.



Esmaeel Moradi
@esmaeel-moradi700



Common CI/CD Tools for Android:

- **Jenkins:** A flexible and powerful automation server (requires infrastructure management).
- **GitHub Actions:** An integrated CI/CD tool within GitHub (simple and integrable).
- **GitLab CI/CD:** Integrated CI/CD solution within GitLab (suited for DevSecOps).
- **Bitrise/CircleCI/Travis CI:** Cloud platforms specialized for mobile development.
- **Fastlane:** A suite of tools for automating release tasks (Signing, Screenshots, Deployments).



Esmaeel Moradi
@esmaeel-moradi700

Sample Workflow in GitHub Actions:



```
# .github/workflows/android_ci.yml
name: Android CI/CD

on:
  push:
    branches:
      - main
      - develop
  pull_request:
    branches:
      - main
      - develop

jobs:
  build_and_test:
    runs-on: ubuntu-latest # or macos-latest for iOS/faster Android builds

    steps:
      - name: Checkout Code
        uses: actions/checkout@v4

      - name: Set up JDK 17
        uses: actions/setup-java@v4
        with:
          distribution: 'temurin'
          java-version: '17'
          cache: 'gradle'

      - name: Grant execute permission for gradlew
        run: chmod +x gradlew

      - name: Run Lint Checks
        run: ./gradlew lintDebug

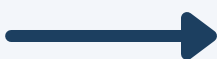
      - name: Run Unit Tests
        run: ./gradlew testDebugUnitTest --info

      - name: Upload Test Reports (Unit)
        uses: actions/upload-artifact@v4
        with:
          name: unit-test-results
          path: app/build/reports/tests/debugUnitTest/

      - name: Build Debug APK
        run: ./gradlew assembleDebug
```



Esmaeel Moradi
@esmaeeil-moradi700



```
- name: Run Instrumented Tests
  uses: reactivecircus/android-emulator-runner@v3 # For running tests on an emulator
  with:
    api-level: 30
    target: default
    arch: x86_64
    profile: Pixel 2
    test-scripts: |
      ./gradlew connectedDebugAndroidTest
- name: Upload Test Reports (Instrumented)
  uses: actions/upload-artifact@v4
  with:
    name: instrumented-test-results
    path: app/build/reports/androidTests/connected/

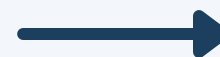
- name: Build Release AAB (on main branch only)
  if: github.ref == 'refs/heads/main'
  run: ./gradlew bundleRelease
  env:
    # Securely pass signing credentials
    SIGNING_KEYSTORE_FILE: ${ secrets.SIGNING_KEYSTORE_FILE }
    SIGNING_KEYSTORE_PASSWORD: ${ secrets.SIGNING_KEYSTORE_PASSWORD }
    SIGNING_KEY_ALIAS: ${ secrets.SIGNING_KEY_ALIAS }
    SIGNING_KEY_PASSWORD: ${ secrets.SIGNING_KEY_PASSWORD }

- name: Deploy to Firebase App Distribution (on main branch only)
  if: github.ref == 'refs/heads/main'
  uses: wzieba/Firebase-Distribution-Github-Action@v1
  with:
    appId: ${ secrets.FIREBASE_APP_ID }
    token: ${ secrets.FIREBASE_TOKEN }
    groups: testers
    file: app/build/outputs/bundle/release/app-release.aab
```



Esmaeel Moradi
@esmaeeil-moradi700

Architect-Level Notes for CI/CD Design:



- **Speed:** The pipeline should be as fast as possible. Use caching for Gradle dependencies and build outputs. Consider parallelization of tests and builds.
- **Stability:** The pipeline must be stable and not "flaky." Fix flaky tests.
- **Security:** Secure management of Credentials (API Keys, Keystore Files) using the CI/CD tool's Secret Management.
- **Test Sharding:** For large applications with hundreds of UI tests, sharding tests across multiple virtual or physical devices is essential to reduce execution time.
- **Different Environments:** The pipeline must be capable of building and deploying versions to different environments (Dev, QA, Staging, Prod), with different configurations (API Endpoints, Feature Flags).
- **Observability:** Test reports and build logs must be easily accessible and analyzable. Integrate with Slack/Teams for notifications.



Esmaeeil Moradi
@esmaeeil-moradi700



- **Rollback Strategy:** Have a plan for rolling back problematic releases in Production.
- **DevSecOps:** Integrate security tools (Static/Dynamic Application Security Testing - SAST/DAST) into the early stages of the pipeline.
- **Version Control Strategy (Git Flow/Trunk Based Development):** The pipeline must align with the team's version control strategy.

With this CI/CD architecture, an Android team can deliver code to users with confidence, speed, and high quality.



Esmaeeil Moradi
@esmaeeil-moradi700

Discuss advanced Debugging techniques: Analyzing ANR Traces, Heap Dumps, CPU Profiles, and Network Traffic.

Advanced debugging goes beyond simply setting breakpoints and stepping through code. In complex applications, we need to analyze data collected from the system to root cause performance and logical issues.



Esmaeel Moradi
@esmaeel-moradi700

1. Analyzing ANR Traces:

- **ANR (Application Not Responding):** As mentioned previously, occurs when the UI Thread is blocked. The Android OS creates **a traces.txt file in /data/anr/** on the device.
- **How to Access:**
 - In Android Studio Profiler (CPU Profiler), if an ANR occurs during a recording, it's usually shown in the Timeline.
 - You can manually pull the file from the device using **adb pull /data/anr/traces.txt** (requires root access in some cases).
 - Crash reporting tools (like Crashlytics or Bugsnag) automatically capture ANRs and report the relevant Stack Traces.



Esmaeel Moradi
@esmaeeil-moradi700

• Analyze the Stack Trace:

- The traces.txt file contains the Stack Trace of all running threads at the time of the ANR.
- **Focus:** Look for the Main Thread (or UI Thread).
- **Follow the Call Stack:** Read the methods from top to bottom (at `com.example.app.MyClass.myMethod(MyClass.java:123)`) to find the method or operation that caused the thread to be blocked.
- **Clues:** Look for patterns like synchronous I/O operations (Network, Disk), heavy computation loops, deadlocks, or excessive locking.
- **Architect's Insight:** Common ANRs include loading large resources on the Main Thread, reading/writing to disk without a separate Thread, or synchronous network calls. Use StrictMode in the development environment to catch these types of issues early.



Esmaeel Moradi

@esmaeel-moradi700

2. Analyzing Heap Dumps:

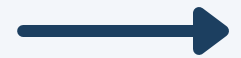


- **Heap Dump:** A snapshot of all objects in the application's memory at a specific point in time. Essential for diagnosing memory leaks and high memory consumption.
- **How to Access:** With Memory Profiler in Android Studio (Dump Java heap) or using adb shell am dumpheap <PID> /data/local/tmp/heapdump.hprof.



Esmaeel Moradi
@esmaeel-moradi700

- **Analysis:**



- **In Memory Profiler, after capturing a Heap Dump:**
 - **Class List:** Sort objects by classes. Look for classes with an unreasonably high number of instances or that occupy a large amount of memory.
 - **Go to Instances:** Inspect suspicious instances.
 - **Analyze References / Path to GC Root:** The most crucial step. This shows the chain of references preventing the object from being garbage collected (GC). The root of these references is usually the point of the memory leak.
 - **Compare Heap Dumps:** Capture two Heap Dumps (one before and one after the suspected leaking operation) and compare them to see objects that should have been collected but are still in memory.
- **Architect's Insight:** LeakCanary (a library from Square) automatically takes Heap Dumps and analyzes the GC Path for common memory leaks, which is an invaluable tool during development. Also, be mindful of large Bitmaps and Context references.



Esmaeel Moradi

@esmaeeil-moradi700

3. Analyzing CPU Profiles:



- **CPU Profile:** A record of your app's CPU activity over time. Useful for identifying performance bottlenecks and reducing Jank (frame drops).
- **How to Access:** Android Studio Profiler (CPU tab).
- **Analysis:**
 - **Recording:**
 - **Trace System Calls:** Detailed system-level information (for JNI, OpenGL analysis).
 - **Sample Java Methods:** Most common and lowest overhead. Periodically samples Stack Traces.
 - **Instrument Java Methods:** Higher overhead but traces all method calls.
 - **Timeline Chart:** Look for spikes in CPU usage and red blocks on the Main Thread.



Esmameil Moradi

@esmaeeil-moradi700

- **Graph Views (Call Chart, Flame Chart, Top Down, Bottom Up):** 
 - **Call Chart:** A time-based graph of method calls. The width of a method indicates its duration, and its depth indicates the Call Stack.
 - **Flame Chart:** A cumulative visualization of sampled Call Stacks. The widest methods consumed the most CPU time.
 - **Top Down:** A list of methods ordered by the time spent in them (Self time and Children time).
 - **Bottom Up:** A list of methods ordered by the time spent in them, but starting from the bottom of the Stack (i.e., methods at the end of the Call Stack).
- **Focus on UI Thread:** Always examine the Main Thread to ensure heavy operations are not performed on it.
- **Architect's Insight:** The CPU Profiler helps you identify inefficient algorithms, heavy loops, and blocking I/O operations. Consider optimizing algorithms, using efficient data structures, and offloading operations to background threads.



Esmaeel Moradi
@esmaeeil-moradi700

4. Analyzing Network Traffic:



- **Network Profiler:** For monitoring your app's network activity. Useful for diagnosing network-related performance issues (latency, high data consumption, excessive requests) or API errors.
- **How to Access:** Android Studio Profiler (Network tab).
- **Analysis:**
 - **Timeline:** Observe network activity over time. Look for spikes in Sent and Received bytes.
 - **Connection View:** Details of each network request:
 - **URL:** Request address.
 - **Method:** Request type (GET, POST, etc.).
 - **Status:** HTTP response code (200 OK, 404 Not Found, 500 Server Error).
 - **Size:** Size of data sent and received.
 - **Time:** Duration of the request completion.
 - **Request & Response Payload:** Full content of the request and response.



Esmaeeil Moradi
@esmaeeil-moradi700



- **Look for Patterns:**
 - **Duplicate Requests:** Is the app sending similar requests too often (Cache Misses)?
 - **Large Responses:** Is the volume of data received from the server more than necessary? (To reduce bandwidth consumption and parsing time).
 - **High Latency:** Are server responses slow?
 - **Server Errors:** Is the app encountering 5xx or 4xx errors from the API?
- **Architect's Insight:**
 - **Caching:** Implement appropriate caching strategies (Disk Cache, Memory Cache) to reduce redundant network requests.
 - **Data Compression:** Use compression to reduce the size of data sent/received (e.g., GZIP).
 - **Batching Requests:** Group multiple small requests into a larger one if possible.
 - **Offline Support:** Design the app to work offline, at least for cached data.
 - **Network Interceptors:** Use Interceptors in libraries like OkHttp to log requests/responses and add custom Headers.



General Debugging Tips:



- **Reproducibility:** Always try to reproduce the issue reliably in the development environment.
- **Incremental Debugging:** Change one thing at a time and observe its effect.
- **Detailed Logging:** Use proper logging (e.g., Timber for Android) to understand the app's flow.
- **Custom View Debugging:** For complex UIs, use the Layout Inspector or even Third-Party tools to visualize layout bounds and recompositions.
- **Native Debugging (LLDB):** For issues in C++/JNI code, use LLDB in Android Studio.

These techniques, as a Senior Architect, will help you delve into and resolve the deepest performance and crash issues in your application.



Esmaeel Moradi
@esmaeel-moradi700



How do you implement A/B Testing or Feature Flag Testing in your application, ensuring testability and reliable data collection?

A/B Testing and Feature Flag Testing are powerful tools for product development and data-driven decision-making. Proper implementation is crucial to ensure testability and data reliability.

1. A/B Testing:

- Goal: To compare two or more versions of a feature (A and B) to determine which version performs best based on predefined metrics (KPIs).
- Example: Comparing the color of a purchase button, or two different user flows for registration.

2. Feature Flag Testing (Toggle):

- Goal: To enable/disable a specific feature at runtime without needing a new app release. This capability is useful for gradual rollout of features, kill switches (emergency disable of a feature), and internal testing.
- Example: Enabling a new feature only for the QA team, or disabling a service if an issue arises.



Esmaeel Moradi

@esmaeel-moradi700

Implementation Architecture for Testability and Reliable Data:

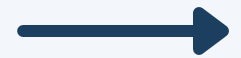
A.Core Architecture:

1-A Central Module or Service for Flag Management:

- Create a FeatureFlagManager (or AbTestManager) class or interface.
- The interface should have methods like
isFeatureEnabled(featureName: Feature),
getVariant(featureName: Feature): Variant,
getExperimentValue(key: String): String.
- Architect's Insight: Use an enum for Features and Variants to prevent typos and make code more readable.
- Example:



Esmaeail Moradi
@esmaeail-moradi700



```
enum class Feature { NEW_PRODUCT_CARD, NEW_ONBOARDING_FLOW }
enum class Variant { CONTROL, TEST_A, TEST_B }

interface FeatureFlagManager {
    fun isEnabled(feature: Feature): Boolean
    fun getFeatureVariant(feature: Feature): Variant
    // Other methods for remote config values
    fun getString(key: String, defaultValue: String): String
    fun getBoolean(key: String, defaultValue: Boolean): Boolean
}

// Production implementation (e.g., uses Firebase Remote Config)
class FirebaseFeatureFlagManager : FeatureFlagManager {
    override fun isEnabled(feature: Feature): Boolean {
        return when (feature) {
            Feature.NEW_PRODUCT_CARD -> Firebase.remoteConfig.getBoolean("new_product_card_enabled")
            // ...
        }
    }
    // ... other implementations
}
```



Esmaeel Moradi
@esmaeeil-moradi700

2-Remote Configuration Fetching:

- Use backend services to manage flags and assign users to A/B groups (e.g., Firebase Remote Config, Optimizely, Split.io, LaunchDarkly).
- Fetch Mechanism: FeatureFlagManager should fetch values from these services at app startup and cache them.
- Fallbacks: Default values (Fallbacks) should be defined for cases where fetching fails.

3-Dependency Injection (DI):

- FeatureFlagManager should be injected into ViewModels, Use Cases, and Composables via DI (like Hilt or Koin). This significantly enhances testability.
- Architect's Insight: With DI, in tests, a Mock or Fake of FeatureFlagManager can be injected, and the behavior of Feature Flags can be fully controlled.



Esmaeel Moradi
@esmaeeil-moradi700

B. Testability:

1-Unit Tests:

- ViewModels or Use Cases that depend on FeatureFlagManager can be tested by injecting a Mock of FeatureFlagManager.
- By mocking, you can easily test various scenarios (e.g., feature is enabled, disabled, user is in group A, user is in group B).
- Example:

```
// Test for a ViewModel that depends on FeatureFlagManager
class MyViewModelTest {
    private lateinit var mockFeatureFlagManager: FeatureFlagManager
    private lateinit var myViewModel: MyViewModel

    @Before
    fun setup() {
        mockFeatureFlagManager = mock()
        myViewModel = MyViewModel(mockFeatureFlagManager)
    }

    @Test
    fun `should show new UI when feature is enabled`() {
        whenever(mockFeatureFlagManager.isFeatureEnabled(FEATURE.NEW_PRODUCT_CARD)).thenReturn(true)
        // Assert that ViewModel state reflects new UI should be shown
        assertEquals(true, myViewModel.showNewCardUI.value)
    }

    @Test
    fun `should show old UI when feature is disabled`() {
        whenever(mockFeatureFlagManager.isFeatureEnabled(FEATURE.NEW_PRODUCT_CARD)).thenReturn(false)
        // Assert that ViewModel state reflects old UI should be shown
        assertEquals(false, myViewModel.showNewCardUI.value)
    }
}
```



Esmaeel Moradi
@esmaeel-moradi700

Integration Tests:



- For testing more complex flows, you can create a Fake or Test implementation of FeatureFlagManager that reads values from a local JSON file or an in-memory Map. This allows you to run integration tests without needing an external server.
- Tip: A custom TestRunner can be created to configure FeatureFlagManager based on test parameters.

3-UI Tests (Espresso / Compose Test APIs):

- UI tests should be able to test both scenarios (feature enabled / feature disabled).
- By injecting the mocked FeatureFlagManager (via DI) into the root Activity/Composable, UI behavior can be tested based on Feature Flags.
- Architect's Insight: A custom Test Rule can be created to set Feature Flags as desired before each UI test.



Esmaeeil Moradi
@esmaeeil-moradi700

C. Reliable Data Collection:



1-Tracking & Analytics:

- Whenever a user is assigned to an A/B Test Variant or a Feature Flag activates a specific state, this information should be sent to the Analytics system (e.g., Firebase Analytics, Google Analytics, Segment).
- Event Logging: Every important user interaction with the new feature (clicks, views, flow completion) should be logged as an Event.
- Example Event: feature_new_onboarding_shown with parameter variant: 'control' or variant: 'test_a'.
- Architect's Insight: Ensure consistency in Event and parameter naming across the team and platforms (Android, iOS, web).
- Data Schema: Define a clear Schema for Analytics data to ensure its validity and analyzability.

2-Monitoring:

- Continuous monitoring of Metrics related to each Variant in Production.
- Use Dashboards (e.g., Looker Studio, Tableau) for visualizing data and comparing Variant performance.
- Note: Pay attention to Statistical Significance in A/B Test results. Do not treat random results as improvements.



Esmaeel Moradi

@esmaeel-moradi700



3-User Identity Management:

- Ensure that user assignment to an A/B group persists over time to maintain a consistent user experience. This can be done by storing the group ID in SharedPreferences or in the User Profile on the backend.
- Architect's Insight: For more complex A/B Tests, an Assignment Service on the backend might be needed to manage groups and report them to the app via API.



Esmaeeil Moradi
@esmaeeil-moradi700

Challenges and Architect-Level Considerations:

- **Code Complexity:** Managing a large number of Feature Flags can make code complex. After an A/B test ends or a feature is fully rolled out, the corresponding Flag should be removed from the code (Clean Up).
- **Performance:** Fetching Flags must be fast and non-blocking. Caching is essential.
- **Security:** Feature Flag values should not contain sensitive information.
- **Developer Experience:** Provide tools for developers (like a Debug screen in the app) to manually enable/disable flags and test different scenarios.
- **Test Environment Parity:** Ensure that test and Production environments are as similar as possible, especially regarding Feature Flags.

By implementing this structured approach, you can ensure the testability of A/B Testing and Feature Flags and use the collected data for intelligent product decisions.



Esmaeel Moradi
@esmaeel-moradi700

Design a Logging and Crash Reporting strategy for a production application, adhering to privacy considerations and efficient error detection.

A robust logging and crash reporting strategy is crucial for any production application. This strategy allows you to identify issues faster, root cause them, and measure their impact on users, all while respecting their privacy.

1. Logging Strategy:

- Goal: Collect information about app behavior and system events for debugging, monitoring, and analysis.
- Log Levels: Standard use of log levels (from highest to lowest priority):
 - ERROR: Severe errors that cause crashes or disrupt app functionality.
 - WARN: Potential issues that might lead to errors in the future (e.g., deprecated API, low resources).
 - INFO: General information about app flow (e.g., "User logged in," "Data received from network").
 - DEBUG: Useful information for debugging during development (e.g., variable values, API responses).
 - VERBOSE: Extremely detailed, only for deep debugging.



Esmaeel Moradi

@esmaeel-moradi700

- **Tools:**

- Timber (recommended for Android): A flexible logging library that allows you to plant different "trees" for different environments.
 - In Debug Build: plant a DebugTree (which sends logs to Logcat).
 - In Release Build: Use a custom CrashReportingTree that only sends WARN and ERROR (and maybe INFO) logs to the crash reporting service and does not publish any DEBUG or VERBOSE logs to Logcat.
- Logcat: The primary tool for viewing logs in Android Studio during development.



Esmaeel Moradi
@esmaeeil-moradi700



- **Privacy Considerations:**

- Never Log PII (Personally Identifiable Information): Sensitive information such as full name, email address, phone number, credit card details, passwords, IP addresses, biometric data, or any data that can directly identify a user.
- Data Masking / Anonymization: If you absolutely need to log sensitive data (which is generally discouraged), Mask or Anonymize it before logging.
- Log Level Filtering: In Production Builds, completely disable DEBUG and VERBOSE logs or send them only to the crash reporting service (without public display in Logcat).
- User Notification: If you collect non-personal data for app performance analysis (like event logs), clearly state it in the app's Privacy Policy.



Esmaeeil Moradi

@esmaeeil-moradi700



- **Efficient Error Detection:**

- Contextual Logging: Logs should contain enough information to understand the context of the error (e.g., anonymized user ID, screen name, operation being performed).
- Error Codes: Use error codes to identify and filter different types of errors.
- Timestamps: Logs should always include precise Timestamps.
- Thread Information: Logs should also indicate the Thread from which the log originated.



Esmaeel Moradi
@esmaeeil-moradi700

2. Crash Reporting Strategy:

- Goal: Automatically collect information about crashes and unexpected errors in end-users' applications.
- Tools (Recommended):
- Firebase Crashlytics: Official Google tool, free, and very powerful.
- Sentry, Bugsnag, Instabug: Third-party options with more advanced features.
- Key Capabilities of Crash Reporting Tools:
- Stack Traces: Automatic collection of full Stack Traces for every crash.
- Contextual Data:
 - Custom Keys/Logs: Ability to add custom Key-Value Pairs (`Crashlytics.setCustomKey(...)`) or custom logs (`Crashlytics.log(...)`) before a crash to understand app state.
 - User ID: (Optional and with privacy in mind) Set an anonymized User ID to track crashes across users (`Crashlytics.setUserId(...)`).
 - Non-Fatal Errors: Report "non-fatal" errors (caught exceptions) that don't crash the app but indicate a problem (`Crashlytics.recordException(...)`).



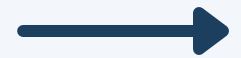
Esmaeel Moradi

@esmaeeil-moradi700

- ANR Reporting: Reporting of ANRs (along with Stack Traces).
- Device Info: Device information (model, Android version, free RAM, free Disk).
- App Info: App version, Build Number.
- Breadcrumbs: Short logs indicating what the user did before a crash.
- Filtering & Grouping: Ability to filter, group, and prioritize crashes.



Esmaeel Moradi
@esmaeeil-moradi700



- **Privacy and Efficiency Considerations (with Crash Reporting):**

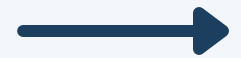
- Default Data Collection: Crash reporting tools typically collect non-identifiable data (like device type, OS version) by default. Clearly state your privacy policy.
- Opt-in/Opt-out (Optional): In some jurisdictions (e.g., GDPR), you might need to give users the option to Opt-in/Opt-out of crash data collection.
- No PII in Custom Data: Re-emphasize never putting any PII in Custom Keys or Custom Logs.
- ProGuard/R8 Mapping: Ensure that ProGuard/R8 Mapping files (mapping.txt) are correctly uploaded alongside the AAB/APK so that obfuscated Stack Traces can be read (De-obfuscated). This is critical for root causing issues.
- Thresholds & Alerts: Set thresholds for crash rates and receive alerts (Email, Slack) when the crash rate exceeds a certain limit.
- Impact Analysis: Analyze the impact of crashes on user experience (e.g., crash-free sessions rate).



Esmaeeil Moradi

@esmaeeil-moradi700

Integrated Logging and Crash Reporting Architecture:



1-A Central Logging Module:

- Create a Wrapper class (e.g., AppLogger) that uses Timber in the background.
- This Wrapper can perform tasks like PII filtering, adding common context (e.g., User Session ID).

```
// In your common logging module
object AppLogger {
    fun debug(message: String, vararg args: Any?) = Timber.d(message, args)
    fun info(message: String, vararg args: Any?) = Timber.i(message, args)
    fun warn(message: String, vararg args: Any?) = Timber.w(message, args)
    fun error(throwable: Throwable? = null, message: String, vararg args: Any?) {
        Timber.e(throwable, message, args)
        // Optionally, send this error to crash reporting for non-fatal
        throwable?.let { Crashlytics.recordException(it) }
    }
}

// In Application class
class MyApplication : Application() {
    override fun onCreate() {
        super.onCreate()
        if (BuildConfig.DEBUG) {
            Timber.plant(Timber.DebugTree())
        } else {
            Timber.plant(CrashlyticsTree()) // Custom tree for production
        }
        // Initialize Crashlytics, etc.
    }
}

// Custom Timber tree for Crashlytics
class CrashlyticsTree : Timber.Tree() {
    override fun log(priority: Int, tag: String?, message: String, t: Throwable?) {
        if (priority == Log.VERBOSE || priority == Log.DEBUG) {
            return // Don't log VERBOSE or DEBUG to Crashlytics
        }
        Crashlytics.log(message)
        t?.let { Crashlytics.recordException(it) }
    }
}
```



Esmaeel Moradi
@esmaeeil-moradi700

2.Using Feature Flags:

- Feature Flags can be used to enable/disable more detailed logging for specific user groups in Production (e.g., for debugging a particular issue for a subset of users).

3.Testing the Strategy:

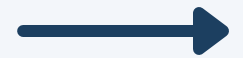
- Test that logs with correct levels and without sensitive information are not displayed in Production Builds.
- Test that crashes and non-fatal errors are correctly sent to the crash reporting service.

By meticulously implementing this strategy, your team can gain deeper insights into the app's state in the hands of real users, effectively root-cause issues, and uphold its responsibility towards user privacy.



Esmaeeil Moradi
@esmaeeil-moradi700

Final Summary for Day 9 Challenge:



With the conclusion of this comprehensive discussion on Android testing strategies, from basic concepts for juniors to deep architectural insights, I hope your understanding of the importance and complexity of software quality assurance has increased. As a Senior Architect, I always emphasize that testing is not just about finding bugs; it's a culture and philosophy that helps you write cleaner, more modular, and more reliable code. Especially in sensitive domains like Web3 and crypto wallets, testing errors can be catastrophic.

Through continuous practice, ongoing learning, and sharing knowledge, you will pave your way in this exciting industry. Stay tuned for the upcoming days of the #30DayAndroidChallenge, where we will delve into even more complex and profound topics! If you have any questions or would like to share your experiences, feel free to connect with me in the comments.



Esmaeel Moradi
@esmaeel-moradi700

✨ These comprehensive answers aim to equip you with the knowledge required for Architect-level and Senior-level interviews, and to make informed architectural decisions in your real-world projects. Good luck. ✨



Esmaeeil Moradi

@esmaeeil-moradi700